FINAL PROJECT REPORT

PDF Merger API

A cloud-native, containerized REST service for merging PDF documents

Course	Bulut & Konteyner Mimarisi (Cloud & Container Architecture)
Author	Demirhan Cakir
Contact	demirhan4856@gmail.com
Repository	github.com/DemirhanCakir/pdf-merger-api
Version	0.1.0
Date	June 2026
License	MIT

FastAPI · PostgreSQL · Amazon S3 (LocalStack) · Docker · Kubernetes · KEDA · Prometheus / Grafana · OpenTelemetry · GitHub Actions CI/CD

Table of Contents

1. Introduction & Project Overview

The **PDF Merger API** is a production-style REST web service that combines multiple uploaded PDF documents into a single merged PDF. While the core functionality — concatenating PDF pages — is intentionally simple, the project's real goal is to demonstrate a complete **cloud-native and container-based architecture** end to end: from a clean application layer, through stateful backing services, into a containerized deployment running on Kubernetes with automated scaling, full observability, and a continuous integration / continuous delivery pipeline.

The service exposes an asynchronous job-based workflow. Clients upload PDF files, receive file identifiers, then submit a merge request referencing those files. The merge runs as a background task; clients poll a job endpoint for status and, once complete, download the merged result. State is persisted in **PostgreSQL** and binary artifacts are stored in an **S3-compatible object store** (LocalStack in development), keeping the API layer stateless and horizontally scalable.

1.1 Project Objectives

- Build a clean, well-tested REST API using a modern asynchronous Python framework.
- Externalize all state to managed backing services so the application tier is stateless.
- Package the application as a minimal, secure, multi-stage Docker image.
- Orchestrate the full system with Docker Compose locally and Kubernetes for production.
- Add horizontal autoscaling driven by real request-rate metrics via KEDA.
- Provide first-class observability: Prometheus metrics, Grafana dashboards, and distributed tracing.
- Automate linting, testing, image build, deployment, and smoke testing through GitHub Actions.
- Validate performance with a realistic k6 load test and documented service-level objectives.

1.2 Project at a Glance

Metric	Value
Application source	~610 lines of Python across 12 modules
Public API endpoints	7 (files, merge, jobs, health, metrics)
Automated tests	35 tests (unit, integration, E2E)
Coverage gate	≥ 70% enforced in CI
Container image	Multi-stage, non-root, ~slim Python 3.12 base
Kubernetes manifests	8 (Deployment, Services, Config, Secret, KEDA, etc.)
CI/CD stages	5 (lint → test → build → deploy → smoke)

2. Functional Requirements & API Design

The API is versioned under the `/api/v1` prefix and follows REST conventions with appropriate HTTP status codes. Uploading returns `201 Created`, starting a merge returns `202 Accepted` (the work is asynchronous), and downloading an unfinished job returns `409 Conflict`. Input is validated with Pydantic models and serialized responses are typed.

2.1 Endpoint Reference

Method	Path	Description	Success
POST	/api/v1/files	Upload a single PDF (multipart)	201
GET	/api/v1/files	List uploaded PDFs (paginated)	200
POST	/api/v1/merge	Start an async merge job	202
GET	/api/v1/jobs/{id}	Poll merge job status	200
GET	/api/v1/jobs/{id}/download	Download the merged PDF	200
GET	/health	DB + S3 readiness probe	200
GET	/metrics	Prometheus exposition format	200

2.2 Validation & Error Handling

- **Content type:** only `application/pdf` (or x-pdf) is accepted; otherwise `415 Unsupported Media Type`.
- **Size limit:** uploads above `MAX_UPLOAD_SIZE_MB` (default 25 MB) are rejected with `413`.
- **Parse check:** the file is parsed with pypdf on upload; an invalid PDF yields `400 Bad Request` and is never stored.
- **Merge constraints:** a request must reference 2–50 existing files; missing IDs return `404 Not Found`.
- **Typed errors:** all error responses share a consistent `{"detail": ...}` schema.

2.3 Example Workflow

```
# 1. upload two PDFs
FID1=$(curl -s -F "file=@a.pdf" :8000/api/v1/files | jq -r .id)
FID2=$(curl -s -F "file=@b.pdf" :8000/api/v1/files | jq -r .id)

# 2. start a merge job (returns 202 + job id)
JID=$(curl -s -X POST :8000/api/v1/merge \
  -H 'Content-Type: application/json' \
  -d "{\"file_ids\":[\"$FID1\",\"$FID2\"]}" | jq -r .id)

# 3. poll until status == completed
curl -s :8000/api/v1/jobs/$JID | jq

# 4. download merged result
curl -OJ :8000/api/v1/jobs/$JID/download
```

3. System Architecture

The system follows a classic stateless-service architecture. The FastAPI tier holds no local state — everything durable lives in PostgreSQL (metadata) or S3 (file bytes). This separation is what makes the deployment safe to scale horizontally: any replica can serve any request, and KEDA can add or remove pods freely. The diagram below shows the full topology as deployed on a Minikube Kubernetes cluster.

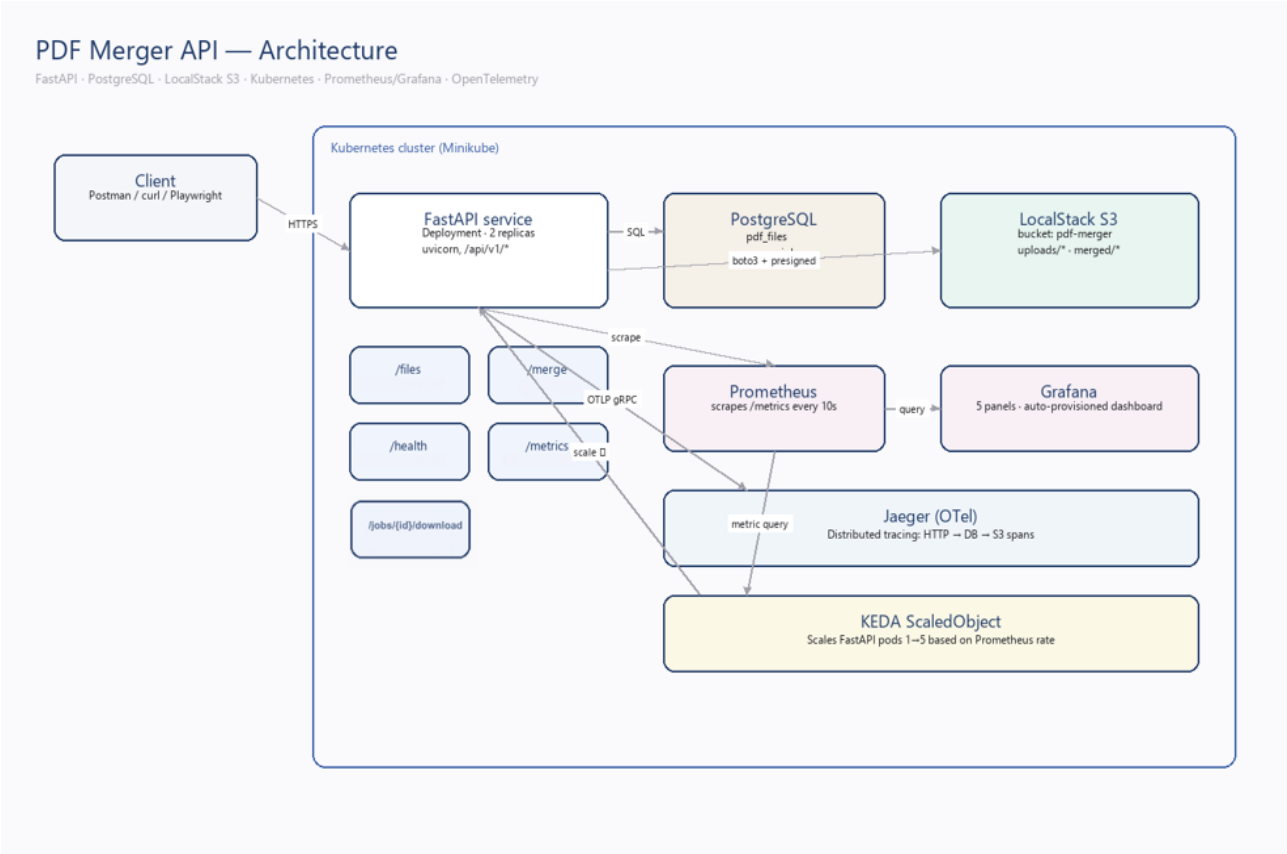

Figure 1 — Deployment architecture: FastAPI service, PostgreSQL, LocalStack S3, Prometheus/Grafana, Jaeger tracing, and KEDA autoscaling inside the cluster.

3.1 Request & Data Flow

- **Upload:** client → FastAPI validates & parses → bytes stored in S3 under `uploads/{id}.pdf` → metadata row written to PostgreSQL.
- **Merge request:** FastAPI verifies all referenced files exist, creates a `pending` job row, and schedules a background task, returning 202 immediately.
- **Background merge:** the worker fetches source bytes from S3, concatenates pages with pypdf, uploads the result to `merged/{id}.pdf`, and flips the job to `completed`.
- **Download:** client polls the job; once complete, the merged PDF is streamed back (and a presigned S3 URL is also offered).

3.2 Design Principles

- **Stateless application tier** — horizontal scaling without sticky sessions.
- **Twelve-factor configuration** — all settings via environment variables / Pydantic Settings.
- **Separation of concerns** — routers, services (PDF, S3), models, and schemas are distinct layers.

- **Fail-safe startup** — the S3 bucket is ensured on boot but a transient failure does not crash the app; it retries on first use.
- **Observability by default** — metrics and optional tracing are wired in at the framework level.

4. Technology Stack

Every dependency is pinned to an exact version in `pyproject.toml` for reproducible builds. The table groups the stack by concern and notes why each choice was made.

Layer	Technology	Role / Rationale
API framework	FastAPI 0.115	Async, type-driven, automatic OpenAPI docs
ASGI server	Uvicorn 0.34	High-performance production server
Validation	Pydantic 2.10	Request/response schemas & settings
Database	PostgreSQL 16	Durable relational metadata store
ORM / migrations	SQLAlchemy 2.0 / Alembic	Typed models; versioned schema
Object storage	S3 / LocalStack 3.8	File bytes; boto3 client + presigned URLs
PDF engine	pypdf 5.1	Pure-Python page concatenation
Containerization	Docker (multi-stage)	Minimal, non-root runtime image
Orchestration	Kubernetes / Minikube	Declarative deployment & scaling
Autoscaling	KEDA	Scale on Prometheus request-rate metric
Metrics	Prometheus + Grafana	Scrape /metrics; visualize 4 key panels
Tracing	OpenTelemetry + Jaeger	Distributed spans: HTTP → DB → S3
CI/CD	GitHub Actions	Lint, test, build, deploy, smoke
Load testing	k6	Ramping-VU scenario with SLO thresholds
E2E testing	Playwright	Drives the Swagger UI in a real browser
Lint / format	Ruff 0.8	Fast linting and formatting

5. Data Model & Persistence

Two tables capture all durable state. **pdf_files** records each uploaded document and its S3 location; **merge_jobs** tracks the lifecycle of a merge operation. Schema is defined twice in a deliberately consistent way: as SQLAlchemy models for the application, and as an Alembic migration (`0001_init`) that owns the production schema. The `create_all()` helper is used only for development and tests.

5.1 pdf_files

Column	Type	Notes
id	UUID	Primary key (uuid4)
filename	VARCHAR(255)	Original client filename
s3_key	VARCHAR(512)	Unique object key in S3
size_bytes	INTEGER	Stored file size
page_count	INTEGER	Parsed page count
uploaded_at	TIMESTAMPTZ	Server timestamp (UTC)

5.2 merge_jobs

Column	Type	Notes
id	UUID	Primary key (uuid4)
status	ENUM job_status	pending / processing / completed / failed
source_file_ids	JSON	Ordered list of source file UUIDs
output_s3_key	VARCHAR(512)	Key of merged result (nullable)
error_message	VARCHAR(1024)	Failure detail (nullable)
created_at	TIMESTAMPTZ	Job creation time
completed_at	TIMESTAMPTZ	Completion time (nullable)

An index on `status` keeps job-status filtering efficient. The job status is modeled as a native PostgreSQL enum, giving database-level integrity to the state machine.

5.3 Job State Machine

```
pending  →  processing  →  completed
          →  failed  (on any error; error_message populated)
```

6. Object Storage (S3)

Binary PDF content is never stored in the database or on a pod's local disk. Instead it lives in an S3 bucket, accessed through a thin boto3 wrapper (`src/services/s3_client.py`). In development and CI the bucket is served by **LocalStack**, a fully local AWS emulator, so the exact same code path works without any cloud account or cost. Switching to real AWS S3 in production is a matter of changing the endpoint and credentials environment variables — no code change.

6.1 Storage Layout & Client Responsibilities

- **uploads/{id}.pdf** — original uploaded documents.
- **merged/{id}.pdf** — merged output keyed by job id.
- **ensure_bucket()** — idempotently creates the bucket on startup (head, then create on 404).
- **presigned_url()** — issues time-limited (1 hour) download links so clients can fetch directly from storage.
- **Path-style addressing + SigV4** — configured explicitly for LocalStack compatibility.
- **ping()** — lists buckets as a lightweight health signal used by the /health probe.

The boto3 client is created lazily and cached with `lru_cache`, and the cache is cleared between tests so each test sees its own isolated bucket configuration.

7. Asynchronous Merge Workflow

Merging is potentially slow (it is CPU-bound and involves several S3 round-trips), so it must never block the HTTP request. The `POST /api/v1/merge` endpoint validates input, persists a `pending` job, schedules the work via FastAPI `BackgroundTasks`, and returns `202 Accepted` right away. The background worker owns its own database session and is fully self-contained.

7.1 The merge function

```
def run_merge_job(job_id):
    job.status = processing; commit()
    blobs = [s3.download(f.s3_key) for f in ordered_sources]
    merged = merge_pdf_bytes(blobs)  # pure, unit-tested
    s3.upload(merged, 'merged/{job}.pdf')
    job.status = completed; job.completed_at = now(); commit()
    # any exception -> status=failed, error_message recorded
```

Crucially, the core `merge_pdf_bytes()` is a **pure function** (list of byte strings in, merged bytes out). This makes it trivial to unit-test in isolation without any database, network, or S3, and keeps the I/O orchestration cleanly separated from the merge logic itself.

Failures are first-class: any exception during the job is caught, the job is marked `failed`, and a truncated error message is stored so clients can see what went wrong without crashing the worker.

8. Containerization (Docker)

The application ships as a **multi-stage Docker image**. A heavier builder stage compiles wheels (including the native psycopg2 dependency); the final runtime stage installs only those pre-built wheels onto a slim Python 3.12 base. This keeps the production image small and free of build toolchains.

8.1 Image Hardening

- **Two-stage build** — build tools (build-essential, libpq-dev) never reach the runtime image.
- **Non-root user** — runs as a dedicated UID 1000 `app` user, not root.
- **Pinned base** — `python:3.12-slim` for a small, predictable footprint.
- **HEALTHCHECK** — container-level probe hits `/health` every 30s.
- **Bytecode & cache hygiene** — `PYTHONDONTWRITEBYTECODE`, no pip cache, apt lists removed.

8.2 Local Orchestration with Docker Compose

`docker-compose.yml` brings up the entire stack with one command: the API, PostgreSQL, LocalStack, Prometheus, Grafana, and Jaeger. Service dependencies use health-condition gating so the API only starts once Postgres and LocalStack report healthy. Grafana is auto-provisioned with its datasource and dashboard mounted read-only, so the full observability stack is live immediately after `compose up`.

Service	Image	Port	Purpose
api	built from Dockerfile	8000	The FastAPI service
postgres	postgres:16-alpine	5432	Metadata store
localstack	localstack:3.8	4566	S3 emulator
prometheus	prom/prometheus:2.55	9090	Metrics scraping
grafana	grafana:11.4	3000	Dashboards
jaeger	jaeger all-in-one:1.76	16686	Trace UI

9. Kubernetes Deployment & Autoscaling

For production-style deployment the system runs on Kubernetes (Minikube locally). The manifests in `k8s/` describe the full application topology declaratively, separating configuration from secrets and exposing the service both internally and via NodePort.

9.1 Manifests

Manifest	Resource	Purpose
deployment.yaml	Deployment	2 replicas, rolling update, probes, limits
service.yaml	Service ×2	ClusterIP + NodePort (30080)
configmap.yaml	ConfigMap	Non-secret app configuration
secret.example.yaml	Secret	DB URL & AWS credentials (template)
postgres.yaml	Deployment/Svc	In-cluster PostgreSQL
localstack.yaml	Deployment/Svc	In-cluster S3 emulator
scaledobject.yaml	KEDA ScaledObject	Request-rate autoscaling
servicemonitor.yaml	ServiceMonitor	Prometheus Operator scrape config

9.2 Production-Readiness Features

- **Rolling updates** with `maxUnavailable: 0` — zero-downtime deploys.
- **Readiness & liveness probes** on `/health` — traffic only to healthy pods; hung pods restarted.
- **Resource requests & limits** — 100m/256Mi requested, capped at 1 CPU / 1Gi.
- **Hardened securityContext** — non-root, no privilege escalation, all Linux capabilities dropped.
- **Config/Secret split** — credentials injected via Secret refs, never baked into the image.

9.3 Autoscaling with KEDA

Rather than scaling on raw CPU, the project uses **KEDA** to scale on a business-meaningful signal: HTTP request rate. The ScaledObject queries Prometheus for the per-second request rate and scales the deployment between **1 and 5 replicas**. When sustained traffic exceeds **10 req/s**, KEDA adds pods; after a 60-second cooldown it scales back down. This directly connects the observability layer to the scaling decision.

```
query: sum(rate(http_requests_total{job="pdf-merger-api"}[2m]))
threshold: 10 req/s  ·  minReplicas: 1  ·  maxReplicas: 5  ·  cooldown: 60s
```

10. Observability

Observability is treated as a core feature, not an afterthought, and spans all three pillars: metrics, dashboards, and distributed tracing.

10.1 Metrics (Prometheus)

The `prometheus-fastapi-instrumentator` exposes a `/metrics` endpoint with standard request counters, latency histograms, and in-progress gauges. Prometheus scrapes this endpoint every 10 seconds; pod annotations advertise the scrape target so discovery is automatic.

10.2 Dashboards (Grafana)

The auto-provisioned Grafana dashboard surfaces four key panels:

- **Request rate** — `sum(rate(http_requests_total[2m]))`.
- **Request rate by endpoint** — broken down per handler.
- **Pod count** — visualizes KEDA scaling in real time.
- **p95 latency** — `histogram_quantile(0.95, ...)`.

10.3 Distributed Tracing (OpenTelemetry → Jaeger)

When `OTEL_ENABLED=true`, the app installs OpenTelemetry auto-instrumentation for FastAPI, SQLAlchemy, and botocore, exporting spans over OTLP/gRPC to Jaeger. A single request can then be traced end to end — HTTP handler → database query → S3 call — making latency attribution straightforward. The feature is import-guarded and a no-op when disabled, so it adds zero overhead in environments that don't need it.

10.4 Structured Logging & Health

Logging is configured centrally at startup with a consistent format, and the `/health` endpoint actively probes both PostgreSQL (`SELECT 1`) and S3 (`list_buckets`), returning `ok` or `degraded`. This same endpoint backs the Kubernetes probes and the Docker HEALTHCHECK.

11. Testing Strategy

The project follows the testing pyramid: many fast unit tests, a solid layer of integration tests against real backing services, and a thin layer of browser-driven end-to-end tests. A coverage floor of **70%** is enforced in CI so the suite cannot silently erode.

Layer	Tooling	Count	What it verifies
Unit	pytest	18	Pure logic: PDF merge, schemas, models, config
Integration	pytest + Testcontainers	12	Real Postgres + LocalStack via the API
E2E	Playwright	5	Swagger UI loads & behaves in a real browser

Counts are approximate per test file; total ≈ 35 test functions.

11.1 Integration Tests with Testcontainers

Integration tests spin up genuine PostgreSQL and LocalStack containers via **Testcontainers**, so the code is exercised against the same engines used in production rather than mocks or SQLite stand-ins. Fixtures recreate the schema and S3 bucket fresh for every test, guaranteeing isolation. In CI these are swapped for service containers (detected via `TEST_DATABASE_URL`) to avoid duplicate startup cost.

11.2 What the Tests Cover

- The pure merge function: correct page totals, rejection of single-file merges, invalid bytes.
- Upload validation: content-type rejection, size limits, unparsable PDFs.
- Full merge lifecycle: upload → merge → poll → completed → download.
- Error paths: unknown file IDs (404), downloading an unfinished job (409).
- Health endpoint reporting ok/degraded; Swagger UI rendering end-to-end.

12. CI/CD Pipeline

A single GitHub Actions workflow (`.github/workflows/ci.yml`) implements a five-stage pipeline that runs on every push and pull request to `main`. Each stage gates the next, so a failure stops the line early.

#	Stage	Actions
1	Lint	ruff check + ruff format --check
2	Test	pytest (unit+integration) w/ service containers, 70% gate, coverage artifact
3	Build	Multi-stage Docker build with GHA layer cache; image saved as artifact
4	Deploy	Spin up Minikube, load image, apply k8s manifests, wait for rollout
5	Smoke	Newman (Postman) API checks + k6 smoke test against the live deploy

The pipeline does far more than run tests: it proves the image actually builds, **deploys to a real Kubernetes cluster**, and serves live traffic correctly via end-to-end smoke tests. The Docker build uses GitHub Actions cache (`type=gha`) to keep rebuilds fast, and the built image is passed between jobs as an artifact so the exact bits that were tested are the bits that get deployed.

12.1 Quality Gates

- Lint and formatting must pass before any test runs.
- Test coverage must stay at or above 70% or the build fails.
- k6 smoke thresholds (error rate, latency) fail the build on regression.
- On smoke failure, API container logs are dumped automatically for debugging.

13. Performance & Load Testing

Performance is validated with a realistic **k6** scenario (`perf/load-test.js`) that mirrors true user behavior: each virtual user uploads two PDFs, starts a merge, then polls until the job completes. The load follows a ramping-VU profile peaking at 50 concurrent users over roughly 100 seconds.

13.1 Load Profile

Stage	Duration	Target VUs
Ramp-up	20 s	1 → 10
Ramp-up	40 s	10 → 50
Sustain	30 s	50
Ramp-down	10 s	50 → 0

13.2 Service-Level Objectives (enforced thresholds)

- **Error rate** < 5% (`http_req_failed`).
- **HTTP latency** p95 < 2000 ms (single request).
- **End-to-end merge flow** p95 < 10000 ms (upload + merge + poll).

These thresholds are encoded in the test itself, so a breach makes k6 exit non-zero and fails CI — performance regressions are caught per commit, not in production.

13.3 Identified Bottlenecks & Mitigations

Bottleneck	Mitigation
pypdf parsing is CPU-bound (pure Python)	25 MB upload size cap; horizontal scale via KEDA
S3 round-trips per merge (download N + upload 1)	Real S3 parallelism in prod; LocalStack only in dev
SQLAlchemy default pool (5) saturates at 50 VUs	Tune pool_size / max_overflow for concurrency
BackgroundTasks share the event loop	KEDA horizontal scaling spreads work across pods

A lighter `smoke-test.js` (1 VU, 10 iterations) runs in CI right after deployment to confirm the live service is healthy in under 30 seconds, separate from the heavier local load test.

14. Security Considerations

- **Least-privilege containers** — non-root UID, dropped capabilities, no privilege escalation.
- **Secrets management** — credentials sourced from Kubernetes Secrets / env vars, never committed; only a `secret.example.yaml` template is in the repo.
- **Input validation** — strict content-type, size, and PDF-parse checks reject malformed or oversized uploads before storage.
- **Pinned dependencies** — exact versions reduce supply-chain drift and make builds reproducible.
- **Time-limited access** — S3 downloads use presigned URLs that expire after one hour.
- **Minimal attack surface** — slim base image with build tools excluded from runtime.

As a course project there are known areas left for hardening — notably authentication/authorization, rate limiting, and per-tenant isolation — which are discussed in Future Work.

15. Challenges & Lessons Learned

- **Local cloud emulation:** LocalStack required explicit path-style addressing and SigV4 signing in the boto3 client to behave like real S3 — a subtle but important configuration detail.
- **Test isolation:** caching the settings and S3 client with `lru_cache` is great for performance but had to be explicitly cache-cleared between tests to avoid leaking state.
- **Async vs. background work:** choosing BackgroundTasks kept the design simple and dependency-free, at the cost of in-process execution — a clear trade-off that KEDA scaling compensates for.
- **Metrics-driven scaling:** wiring KEDA to a Prometheus query (rather than CPU) made scaling behavior intuitive and directly observable on the Grafana pod-count panel.
- **Pipeline realism:** getting CI to actually deploy to Minikube and smoke-test the live service caught issues that unit tests alone never would.
- **Separation pays off:** keeping the merge as a pure function made the most important logic the easiest part to test.

16. Future Work & Conclusion

16.1 Future Work

- **Authentication & authorization** (API keys / OAuth2) and per-user file ownership.
- **Dedicated task queue** (Celery / RQ / Arq) to move merging out of the web process entirely.
- **Rate limiting** and request quotas to protect the service under abuse.
- **Lifecycle policies** to expire old uploads and merged outputs from S3.
- **Richer PDF operations**: page ranges, reordering, rotation, and compression.
- **Alerting** via Prometheus Alertmanager on error-rate and latency SLO breaches.

16.2 Conclusion

The PDF Merger API delivers a small, focused feature on top of a complete, production-minded cloud-native platform. Every layer expected of a modern containerized service is present and working together: a clean, well-tested FastAPI application; externalized state in PostgreSQL and S3; a hardened multi-stage container image; declarative Kubernetes deployment with metrics-driven KEDA autoscaling; full observability through Prometheus, Grafana, and OpenTelemetry tracing; and a five-stage CI/CD pipeline that lints, tests, builds, deploys, and smoke-tests on every change.

The result is a system whose architecture — not its PDF merging — is the deliverable: it demonstrates how to take a simple service and run it the way real cloud software is built, deployed, scaled, observed, and continuously shipped. The same patterns scale directly to far more complex workloads.

Repository: github.com/DemirhanCakir/pdf-merger-api · **License:** MIT · **Author:** Demirhan Cakir (demirhan4856@gmail.com)